

Gibbs Sampling

M08: Lecture 6 (Pages 18-20)

Callum Arnold

22 July, 2021

Contents

Multivariate targets	1
Combining Markov kernels	1
Gibbs sampling	2
Sequential (component) scan	2

Multivariate targets

- Often want to model multivariate situation
- Hamiltonian Monte Carlo allows us to specify a Markov chain made up of multiple random variables

Combining Markov kernels

- Suppose we have 2 MH algorithms
 - Each is a Markov chain with associated transition probability/density matrix

$$\begin{array}{ll} P_1 & MH_1 : y = x_n + \epsilon_n, \quad \epsilon_n \sim \text{Unif}[-\sigma_1, \sigma_1] \quad \text{where: } \sigma_1 = 5.0 \\ P_2 & MH_2 : y = x_n + \epsilon_n, \quad \epsilon_n \sim \text{Unif}[-\sigma_2, \sigma_2] \quad \text{where: } \sigma_2 = 0.5 \end{array}$$

- Let X_0 = initial state
 - $X_1 \leftarrow \text{use } MH_1$
 - $X_2 \leftarrow \text{use } MH_2$
 - $X_3 \leftarrow \text{use } MH_1$
 - $X_4 \leftarrow \text{use } MH_2$
 - ...
 - Known as the sequential scan algorithm
- This is OK because we can think of the Markov chain going from $X_0 \rightarrow X_2 \rightarrow X_4 \rightarrow \dots$
 - $\pi^T P = \underbrace{\pi^T P_1}_{\pi^T} P_2 = \underbrace{\pi^T P_2}_{\pi^T} = \pi^T$ which preserves the global balance
- This is pretty similar to Gibbs sampling (applying 2 Markov chains sequentially).
- Another system would be to create a probability vector and randomly choose which kernel to advance with

- $\boldsymbol{\alpha} = \alpha_1, \dots, \alpha_m$
- $\mathbf{R} = \sum_{i=1}^m \alpha_i \mathbf{P}_i$
- This is known as the random scan, which can also be implemented in Gibbs sampling

Gibbs sampling

Sequential (component) scan

- Suppose we have a multivariate target $\mathbf{x} = x_1 \times \dots \times x_m$
- Our target density is $f(\mathbf{x})$ and we want to calculate $E_f[h(\mathbf{x})]$
- We assume we can sample from full conditional distributions $f_i(x_i | \mathbf{x}_{-i}^{\text{cur}})$ (i.e. given all elements except i)
- You could try and use the standard MH with random walk
 - $\mathbf{y}_n = \mathbf{x}_n + \boldsymbol{\epsilon}_n, \quad \boldsymbol{\epsilon}_n \sim \text{Multivariate } \mathcal{N}(\mathbf{0}, \boldsymbol{\sigma})$
 - Doesn't work well in high dimensions
 - * Reject a lot unless you have a great sigma matrix
- In practice, we often update one component at a time
 - $MH_1: \quad \mathbf{x} = (\underbrace{x_1}_{\substack{\text{update} \\ \text{comp 1}}}, x_2, \dots, x_m)$
 - $\mathbf{x}_{\text{current}} \rightarrow \mathbf{x}_{\text{proposed}}$ but these two only differ in the 1st component
 - Evaluate acceptance probability etc, and repeat for $x_2 \dots x_m$
 - * Now you are sampling at time $t+1$ for x_1 and t for everything else (you haven't updated the other values yet!)
 - Sample $x_2^{(t+1)} \sim f_2(x_2 | x_1^{(t+1)}, x_3^{(t)}, \dots, x_m^{(t)})$
 - ...
 - Sample $x_m^{(t+1)} \sim f_m(x_m | \mathbf{x}_{-m}^{(t)})$
- To see why this works, let's quickly remember our MH acceptance ratio in continuous form
 - Evaluate the target density at the proposed value ($f(\mathbf{y})$) vs old value ($f(\mathbf{x})$), multiplied by the ratio of the proposed densities (reversed)

$$a(\mathbf{x}, \mathbf{y}) = \min \left\{ \frac{f(\mathbf{y})q(\mathbf{x}|\mathbf{y})}{f(\mathbf{x})q(\mathbf{y}|\mathbf{x})}, 1 \right\}$$

- Let's write out our MH acceptance ratio now

$$\begin{aligned} a(\mathbf{x}^{\text{cur}}, \mathbf{x}^{\text{new}}) &= \min \left\{ \frac{f(\mathbf{x}^{\text{new}})q(\mathbf{x}^{\text{cur}}|\mathbf{x}^{\text{new}})}{f(\mathbf{x}^{\text{cur}})q(\mathbf{x}^{\text{new}}|\mathbf{x}^{\text{cur}})}, 1 \right\} \\ &= \min \left\{ \frac{f(x_i^{\text{new}}, \mathbf{x}_{-i}^{\text{cur}})f_i(x_i^{\text{cur}}|\mathbf{x}_{-i}^{\text{cur}})}{f(x_i^{\text{cur}}, \mathbf{x}_{-i}^{\text{cur}})f_i(x_i^{\text{new}}|\mathbf{x}_{-i}^{\text{cur}})}, 1 \right\} \\ &= \min \left\{ \frac{f(\mathbf{x}_{-i}^{\text{cur}})}{f(\mathbf{x}_{-i}^{\text{cur}})}, 1 \right\} \\ &= 1 \end{aligned}$$

- We can see that because we have the full conditional $(f_i(x_i|\mathbf{x}_{-i}^{\text{cur}}))$, it always accepts, so we don't need to compute the acceptance ratio